

ECE391
Computer System Engineering
Lecture 5

University of Illinois at Urbana-Champaign

Spring 2024

Lecture Topics

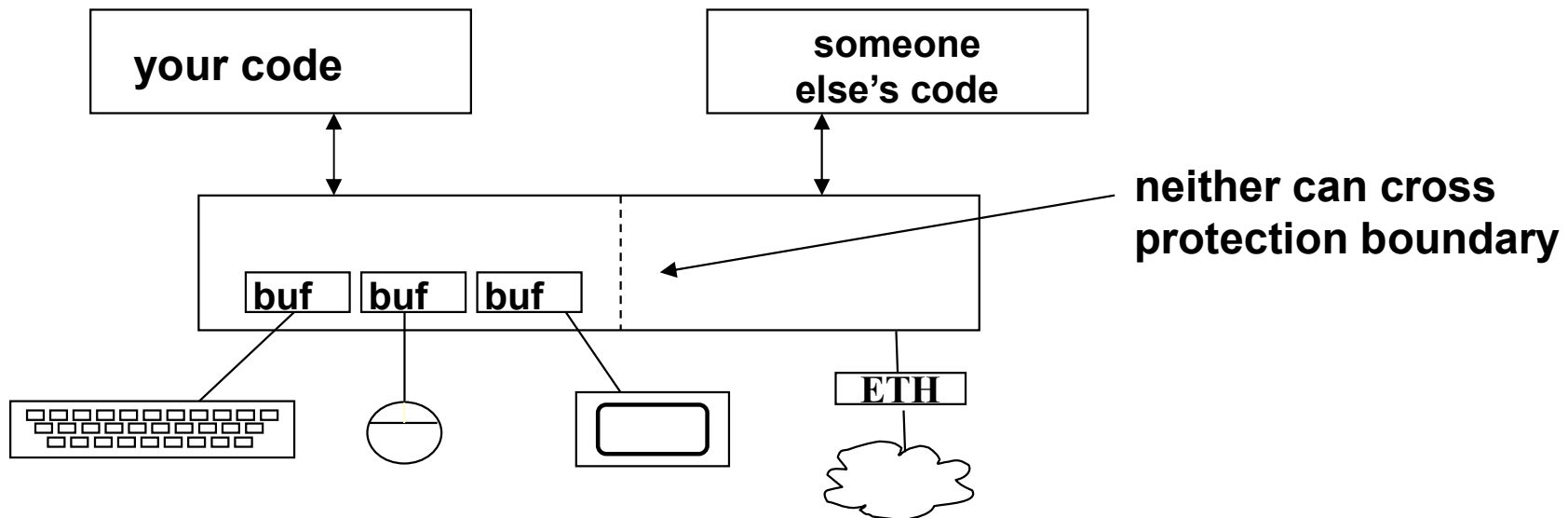
- Role of system software
- Device I/O models
- Interrupts, exceptions, and system calls
- Processor support for interrupts

Role of System Software (1)

- System software serves three purposes

Role of System Software (2)

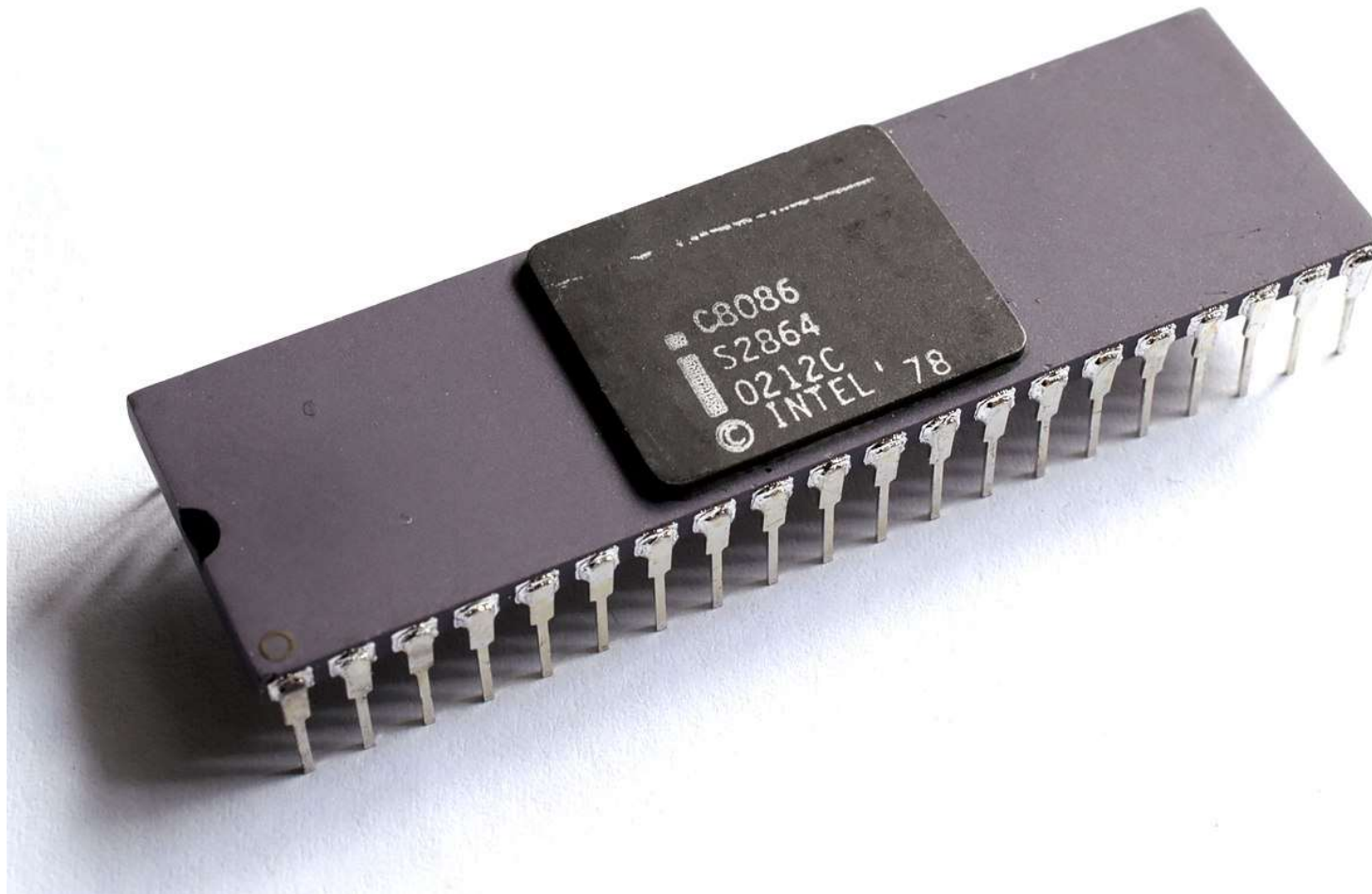
- hide fundamentally asynchronous nature of processor/device interaction
- provide simpler and more powerful interfaces (integrated w/protection)



History

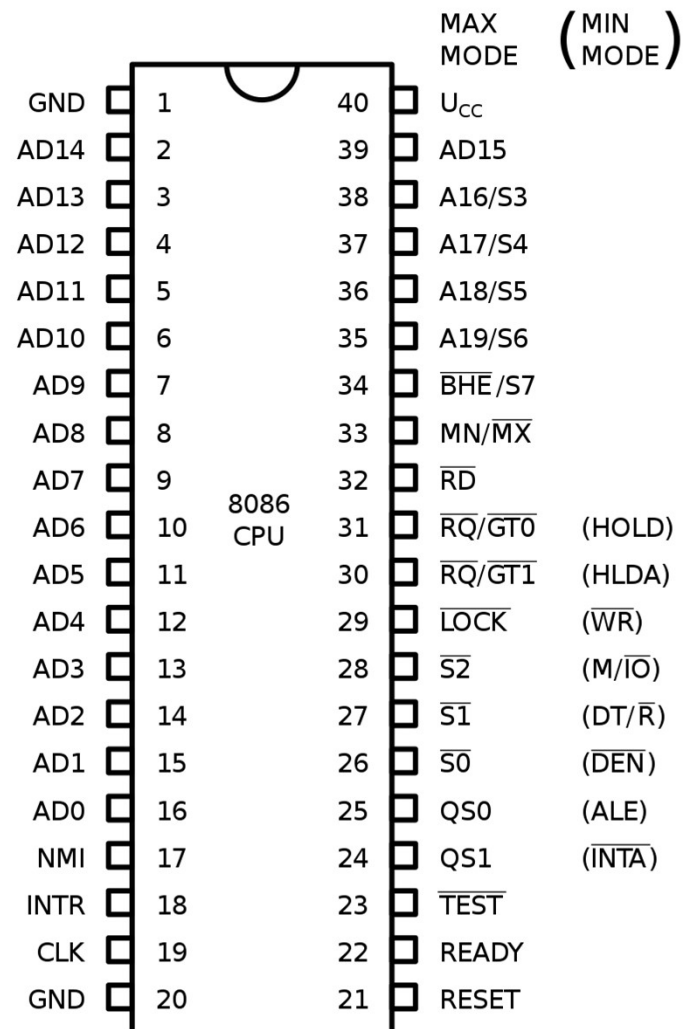
- **1971:** Intel 4004 (4-bit CPU)
- **1972:** Intel 8008 (8-bit CPU)
- **1974:** Intel 8080 (8-bit CPU)
- **1976:** Intel 8085 (8-bit CPU)
 - Binary-compatible with 8080, a few new instructions
- **1978:** Intel 8086 (16-bit CPU)
 - Easy to port programs from 8080/8085
 - Used in IBM PC
- **1982:** Intel 80286 (16-bit CPU)
 - Used in IBM PC/AT
- See <http://www.vgamuseum.info/index.php/cpu>

Intel 8086



Device I/O

- ADX is the (multiplexed) address and data bus
- How do we talk to outside world?
- External devices look like memory:
 - Read a device register as if it were a memory location
 - See ECE 220 lecture on I/O registers for refresher



Device I/O

- How does a processor communicate with devices?
- Two possibilities
 - port I/O — use special instructions and a separate I/O port address space
 - memory-mapped I/O — use loads/stores and dedicate part of the memory address space to I/O
- x86 originally used only port I/O
 - but when used in PC, needed a good interface to video memory
 - solution? put card on the bus, claim memory addresses!
 - now uses both, although ports are used for legacy devices

- I/O instructions have not evolved since 8086
 - 16-bit port space
 - byte addressable
 - little-endian (looks like memory)
 - instructions
 - IN port, dest.reg
 - OUT src.reg, port
 - the register operands are NOT general-purpose registers
 - all data to/from AL (8 bits), AX (16 bits), or EAX (32 bits)
 - port is either an 8-bit immediate (not 16!) or DX (16 bits)

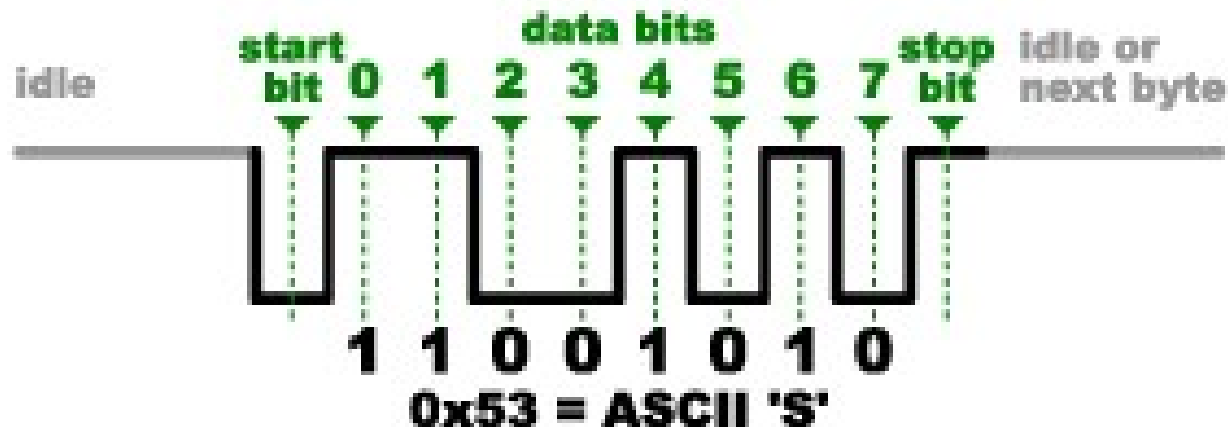
Memory and I/O Address Space

34		BME/SI	
33		MN/MX	
32		RD	
31		RQ/GT0	(HOLD)
30		RQ/GT1	(HLDA)
29		LOCK	(WR)
28		S2	(M/IO)
27		S1	(DT/R)
26		S0	(DEN)
25		...	...

- M/~IO pin tells external hardware whether address is in memory space or I/O space
- Most architectures have memory-mapped I/O only
 - Motorola 6800 (contemporary of 8080) had memory-mapped I/O only

Device Example: IBM PC Serial Port

- Uses NS8250 mapped at address 0x3F8 in I/O space
- **Software view:** sends and receives a byte at a time
- **Physical view:** sends bytes one bit at a time on line
- Asynchronous (no clock signal)
 - Both sides must use approximately same rate
- One of the oldest ways of transmitting data



Device Example: IBM PC Serial Port

- **RBR** (Receive Buffer Register):
contains received data byte
- **THR** (Transmitter Holding Register):
write byte to send here
- Reading I/O addr 0x3F8+0 accesses RBR
- Writing I/O addr 0x3F8+0 accesses THR
- **LSR** (Line Status Register) at address 0x3F8+5
 - **DR** (bit 0): data ready in RBR, reset to 0 after reading RBR
 - **THRE** (bit 5): THR empty, reset to 0 on write to THR

Device Example: IBM PC Serial Port

- Assume you have C functions `inb` and `outb` that do an I/O read and I/O write
- Write a simple `getchar` and `putchar` function

```
static inline void outb(uint16_t port, uint8_t c) {  
    asm volatile ("outb %0, %1" : : "a"(c), "Nd"(port) : "memory");  
}
```

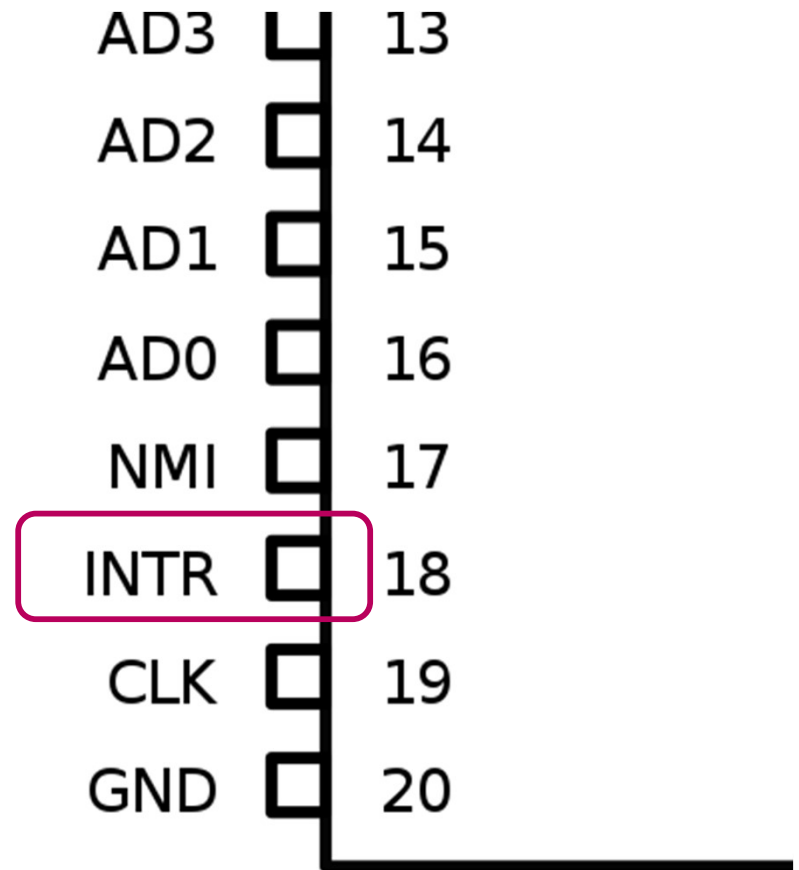
```
static inline uint8_t inb(uint16_t port) {  
    uint8_t c;  
    asm volatile ("inb %1, %0" : "=a"(c) : "Nd"(port) : "memory");  
    return c;  
}
```

Polling-Based I/O

- In the **polling-based I/O model**,
- What is the downside of this?
- Extra operations when nothing happens
- Can miss events if not polling often enough
 - E.g. new byte arrives before CPU reads RBR

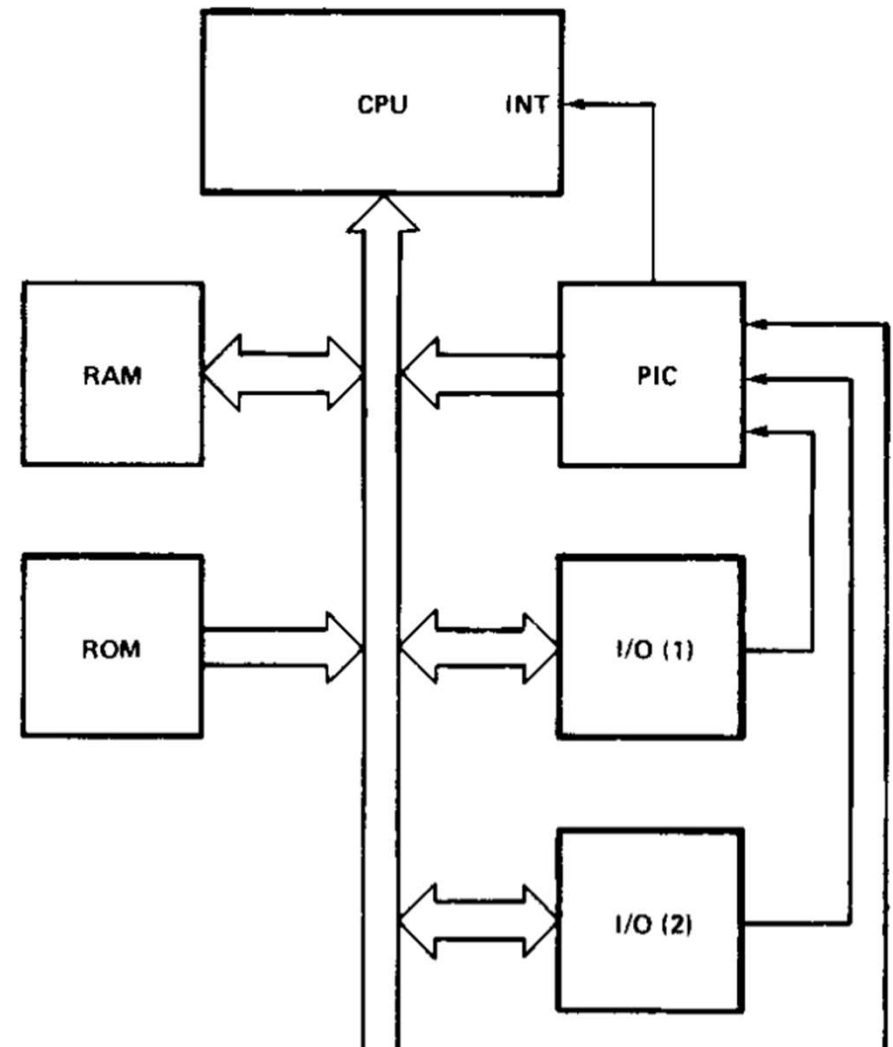
Interrupt-Based I/O

- It would be great if device could interrupt CPU when important state change happens
 - E.g. when data available
- Modern CPUs allow external devices to do this via interrupt request pin
- Called



Multiple Interrupt Lines

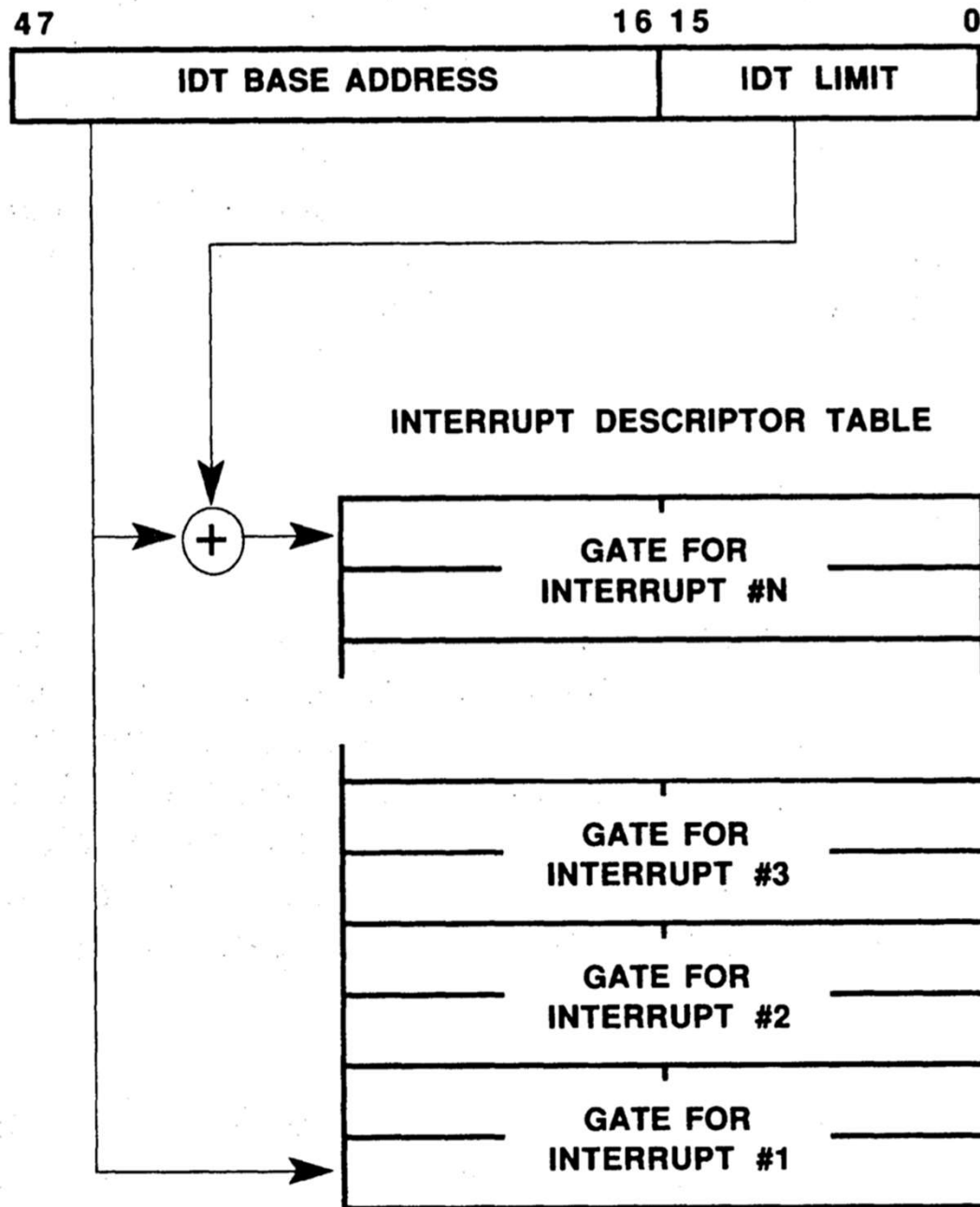
- Programmable Interrupt Controller (PIC) collects all interrupt lines
- Raises INTR for CPU
- Puts ID of highest priority interrupt on data bus
- CPU uses this to determine which device is interrupting
- We will cover PICs in more detail later



Interrupt Descriptor Table

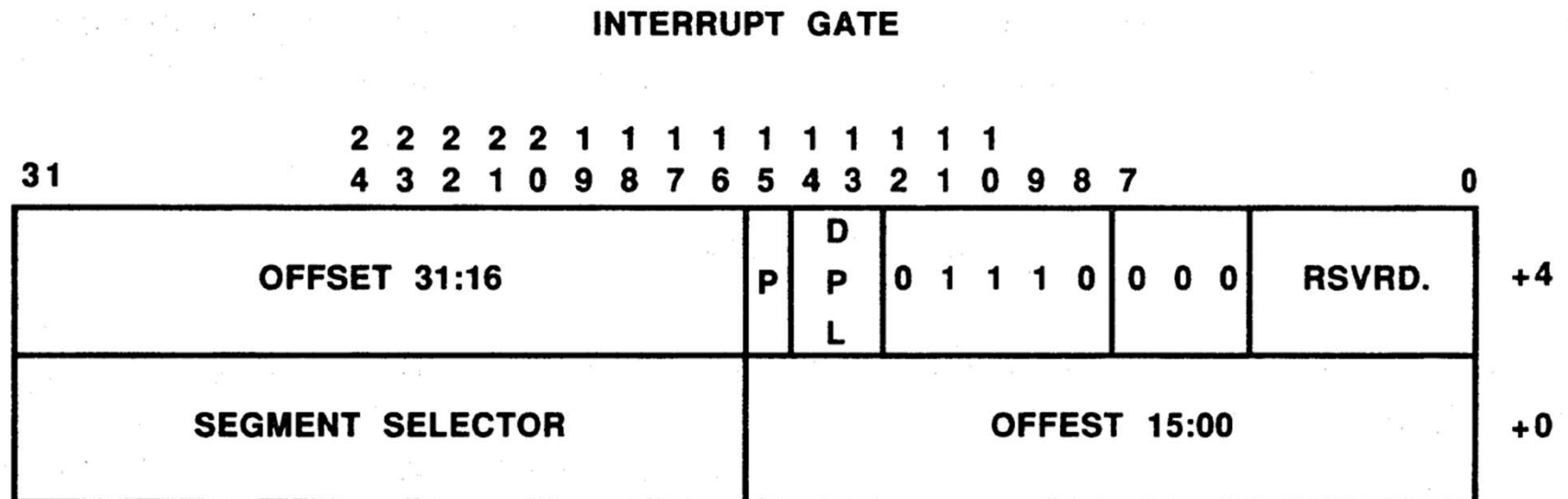
- When an interrupt arrives, CPU drops what its doing and start executing an
 - Also called an *interrupt service routine*
- Need a way to tell CPU where to jump:
Interrupt Descriptor Table (IDT) in x86
 - *Interrupt vector table* in other architectures
- IDT register (IDTR) has base address of IDT and the number of entries
 - *Not* a general-purpose register
 - Load using `lidt` instruction

IDTR REGISTER



IDT Entries (Gates)

- Tells CPU where to start execution on interrupt
- Offset (32-bits) is address of interrupt handler
 - Actually, it's the offset within the code segment, but we will cover IA-32 segmentation later



Interrupt Handling on x86 (Very Simplified)

- Before each instruction, CPU checks for interrupts
- If interrupt i is raised, CPU fetches gate i from IDT
- CPU sets `CS` to offset specified in interrupt gate i
 - This starts executing the interrupt handler
- Handler executes `iret` instr. when done to resume execution from where it was interrupted

CPU, Interrupted

On entry to handler

On return from handler

- *Depending on architecture, CPU does some/most of this context management*

Interrupt Handling on x86 (More Detail)

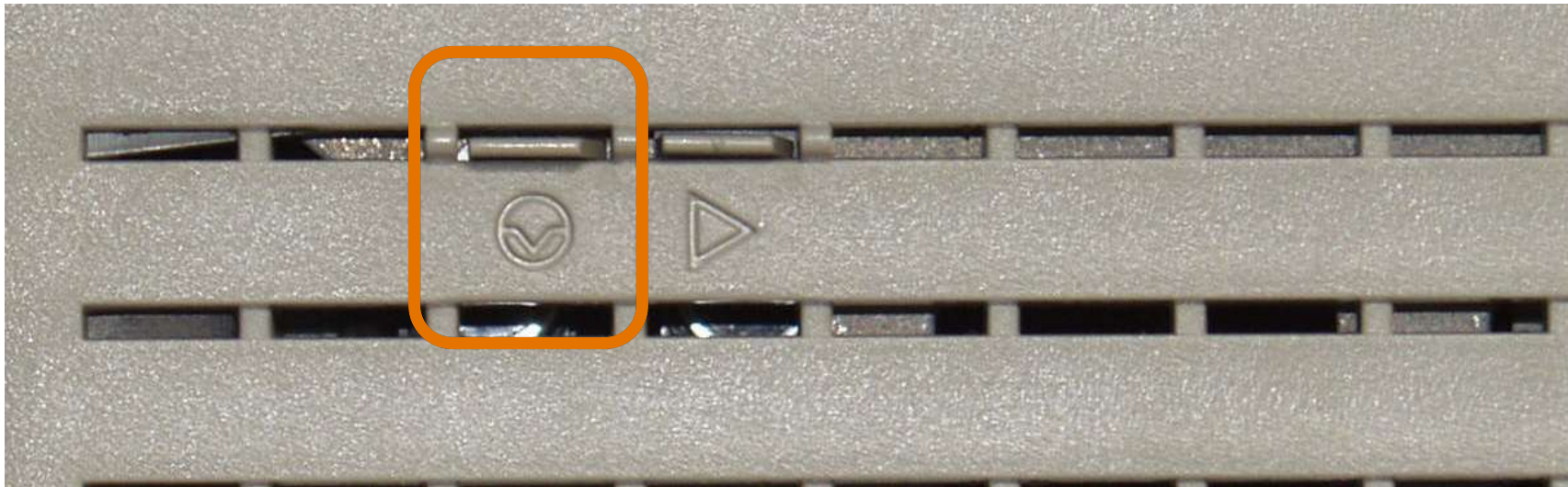
- If interrupt i is raised, CPU fetches entry i from IDT
- Interrupt gate specifies *privilege level* of handler
 - Kernel (incl. interrupt handlers) run at highest privilege
 - User code runs at lowest privilege level
- If interrupt privilege level higher than current:
 - CPU switches to handler's privilege level
 - CPU switches to stack for higher privilege level
- CPU to new stack
- Return from interrupt (`iret` instr.) restores above

Stop Interrupting Me

- Can temporarily disable interrupt processing via interrupt flags (IF) bit in EFLAGS register
 - `cli` instr. clears IF to 0, `sti` sets IF to 1
 - Can also disable (mask) specific interrupts at PIC
 - More on this when we cover PICs
- Some interrupts cannot be disabled: these are called
- Can be used as debugger break, signal irrecoverable fault, etc.

“Programmer’s Switch”

- The “Programmer’s Switch” on early Macintosh computers generated an NMI
 - Could be used to print memory, dump registers



Exceptions

- Interrupts were such a great idea, that designers started using same mechanism for internal events
- An *exception* is an exceptional internal condition encountered during execution
- Why have exceptions?
- Instead of halting, we can recover from some errors
- If user program causes an error, the OS terminates it: *rest of system is not affected!*
 - Actually, some exceptions generate *signals* to program

An Exceptional Idea

- CPUs might not implement entire instruction set
 - Example: CPU does not implement floating point instructions
- Use illegal instruction exception to provide a software implementation of unimplemented instruction
 - Exception handler examines offending instruction
 - If it knows how to do it, it executes a software implementation of the instruction
- Slow, but better than nothing

Check Your Privilege

- All but the simplest CPUs support multiple privilege modes of execution
- Two is very common: *privileged* and *unprivileged*
 - Privileged mode also called *supervisor* mode or *kernel* mode
- Intel x86 has four *rings*: 0 is highest privilege
 - Most OSes (incl. Linux) use only 0 and 3 (unprivileged)
- With virtualization and trusted execution environments (beyond scope of this class), multiple levels are making a comeback

It's Been a Privilege to Know You

- OS runs in privileged mode,
user programs run in unprivileged mode
- Some operations only allowed in privileged mode
 - Can you guess one we've already talked about?

System Calls

- Interrupts and exceptions provide a way to switch to higher privilege mode
 - In some CPUs, this may be the only way
- Interrupts can be triggered *intentionally* as a way for unprivileged program to switch to kernel
 - `int` instruction triggers interrupt from software
- How is this different from a function call?

Linux System Calls

- Programs invoke Linux services via system calls
- Invoked using `int $0x80` instruction
- contains system call number

1. `exit`

4. `write`

7. `waitpid`

2. `fork`

5. `open`

8. `creat`

3. `read`

6. `close`

9. `link`

- Parameters in `%ebx`, `%ecx`, `%edx`, `%esi`, `%edi`, `%ebp`
- Return value in `%eax`
- System call wrapped in function provided by `libc`

Interrupt Descriptor Table (IDT)

0x00–0x1F defined by Intel	0x00	division error	
	⋮		
	0x02	NMI (non-maskable interrupt)	
	0x03	breakpoint (used by KGDB)	
	0x04	overflow	
	⋮		
	0x0B	segment not present	
	0x0C	stack segment fault	
	0x0D	general protection fault	
	0x0E	page fault	
	⋮		
0x20–0x27 primary 8259 PIC	0x20	IRQ0 — timer chip	example of possible settings
	0x21	IRQ1 — keyboard	
	0x22	IRQ2 — (cascade to secondary)	
	0x23	IRQ3	
	0x24	IRQ4 — serial port (KGDB)	
	0x25	IRQ5	
	0x26	IRQ6	
	0x27	IRQ7	
0x28–0x2F secondary 8259 PIC	0x28	IRQ8 — real time clock	
	0x29	IRQ9	
	0x2A	IRQ10	
	0x2B	IRQ11 — eth0 (network)	
	0x2C	IRQ12 — PS/2 mouse	
	0x2D	IRQ13	
	0x2E	IRQ14 — ide0 (hard drive)	
	0x2F	IRQ15	
0x30–0x7F	⋮	APIC vectors available to device drivers	
0x80	0x80	system call vector (INT 0x80)	
0x81–0xEE	⋮	more APIC vectors available to device drivers	
0xEF	0xEF	local APIC timer	
0xF0–0xFF	⋮	symmetric multiprocessor (SMP) communication vectors	

- x86 uses a single vector table **Interrupt Descriptor Table**
 - hold vectors for interrupts, exceptions, and system calls

Interrupts, System calls, and Exceptions

Characteristics of *interrupts, system calls, and exceptions* with respect to a program being executed

Type	Generated by	Asynchronous	Unexpected
Interrupts	external device		
Exceptions	invalid opcode or operand		
System calls	deliberate, via INT instruction		